

DATA.md — Documentation Complète des Données

Projet : Portail RH ArabSoft **Contexte** : PFE GLSI 3ème année — ISTIC, Université de Carthage (2026) **Étudiant** : Rayen — Encadré par ArabSoft, Tunis **Date de génération** : 2026-04-24

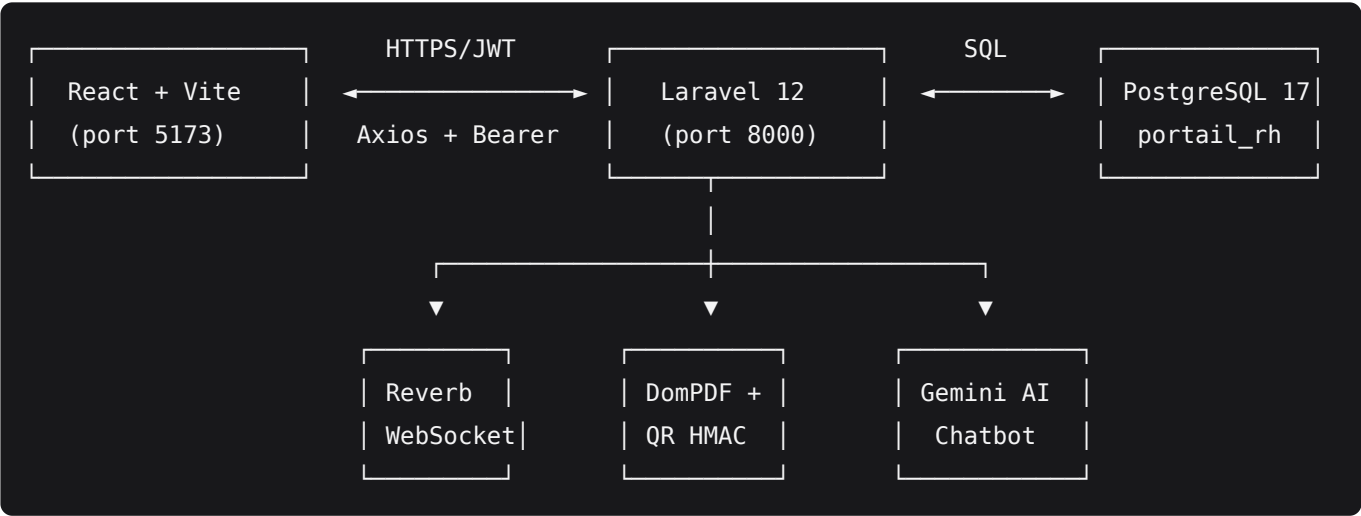
Table des matières

1. Stack technique & architecture
 2. Acteurs et rôles
 3. Schéma de base de données
 4. Enums et valeurs métier
 5. Règles métier
 6. API REST
 7. Modèles Eloquent
 8. Frontend — pages et composants
 9. Fichiers et stockage
 10. Intégrations externes
 11. Seeders et données de test
 12. Variables d'environnement
-

1. Stack technique & architecture

Couche	Technologie	Version
Backend	Laravel	12 (PHP 8.2)
Base de données	PostgreSQL	17 (port 5432)
Auth	tymon/jwt-auth	2.3.0
Frontend	React + Vite	19
Router	React Router DOM	—
HTTP Client	Axios	—
WebSocket	Laravel Reverb	—
PDF	DomPDF	—
IA	Google Gemini 2.5 Flash	—

Architecture simplifiée



2. Acteurs et rôles

Le portail distingue **3 rôles** avec des périmètres distincts :

Rôle	Code DB	Portée des données	Actions
Employé	<code>employe</code>	Ses propres demandes, son profil, son solde de congés, ses messages	Créer demandes, discuter via chatbot, envoyer messages, consulter ses notifications
Chef	<code>chef</code>	Demandes (type <code>conge</code> / <code>autorisation</code>) de son équipe (<code>chef_id = user.id</code>)	Valider/refuser demandes équipe + droits employé
Admin	<code>admin</code>	Tout : tous les utilisateurs, toutes les demandes, dashboard global, resets mot de passe	Valider toutes demandes, créer/modifier/supprimer employés, gérer les soldes de congés

Contrainte DB : `CHECK (role IN ('employe','chef','admin'))` . Attention : certaines migrations utilisent `enum('employee', 'chef', 'admin')` — la contrainte PostgreSQL effective est `employe` (sans le `e` final) conformément à `CLAUDE.md` .

3. Schéma de base de données

8 tables principales + `personal_access_tokens` (auto-générée par Laravel Sanctum, non utilisée activement).

3.1 `users` — Comptes utilisateurs

Colonne	Type	Nullable	Contrainte	Description
id	bigint (PK)	non	—	Identifiant
nom	varchar(100)	non	—	Nom de famille
prenom	varchar(100)	non	—	Prénom
email	varchar(150)	non	UNIQUE	Email de connexion
password	varchar(255)	non	hashed (bcrypt)	Mot de passe
role	enum	non	employe chef admin	Rôle
chef_id	bigint	oui	FK → users.id (onDelete)	Chef hiérarchique
departement	varchar(100)	oui	—	Département
poste	varchar(100)	oui	—	Poste occupé
telephone	varchar(20)	oui	—	Téléphone
photo	varchar(255)	oui	—	Chemin photo (avatars/{id}_{ts}.ext)
created_at / updated_at	timestamp	non	—	Auditabilité

Relations Eloquent : chef(), employes(), congésSolde(), demandes() **Accesseur :** photo_url
→ URL publique absolue de la photo

3.2 demandes — Demandes RH

Colonne	Type	Nullable	Contrainte	Description
id	bigint (PK)	non	—	Identifiant
employee_id	bigint	non	FK → users.id (cascade)	Auteur de la demande
chef_id	bigint	oui	FK → users.id	Chef validateur
admin_id	bigint	oui	FK → users.id	Admin validateur
type	enum	non	conge autorisation pret situation document	Type
statut	enum	non (def. en_attente)	en_attente valide_chef approuvee refusee approuvee_direct	Statut
date_debut	date	oui	≥ today (sauf document)	Début
date_fin	date	oui	≥ date_debut	Fin (congé)
montant	decimal(10,2)	oui	—	Montant prêt (DT)
duree	varchar	oui	3 mois 6 mois 12 mois 24 mois	Durée remboursement
motif	varchar	oui	—	Sous-type / raison
type_document	varchar(100)	oui	—	Type document demandé
commentaire	text	oui	—	Commentaire employé
commentaire_chef	text	oui	—	Commentaire chef
commentaire_admin	text	oui	—	Commentaire admin
piece_jointe	varchar(255)	oui	—	Chemin justificatif (demandes/{id}_{ts}.ext)

Colonne	Type	Nullable	Contrainte	Description
created_at / updated_at	timestamp	non	—	Auditabilité

Relations : `employee()` , `chef()` , `admin()` **Accesseur :** `piece_jointe_url`

3.3 notifications — Notifications in-app

Colonne	Type	Nullable	Description
id	bigint (PK)	non	—
user_id	bigint	non	FK → users.id (destinataire)
demande_id	bigint	oui	FK → demandes.id (cascade)
message	text	non	Message affiché
lu	boolean	non (def. false)	Lue ?
created_at	timestamp	non (useCurrent)	Date création

Spécificité : pas de `updated_at` — dans le modèle, `public $timestamps = false` .

3.4 messages — Messagerie interne

Colonne	Type	Nullable	Description
id	bigint (PK)	non	—
expediteur_id	bigint	non	FK → users.id
destinataire_id	bigint	non	FK → users.id
contenu	text	non	Texte
lu	boolean	non (def. false)	Lu ?
created_at	timestamp	non (useCurrent)	—

3.5 documents — Documents administratifs générés

Colonne	Type	Nullable	Description
id	bigint (PK)	non	—
demande_id	bigint	oui	FK → demandes.id
employee_id	bigint	non	FK → users.id
nom_fichier	varchar(255)	non	Nom affiché
chemin_fichier	varchar(500)	non	Chemin stockage
type_doc	varchar(100)	non	Type (attestation, bulletin...)
created_at	timestamp	non (useCurrent)	—

3.6 `conges_soldes` — Soldes de congés par année

Colonne	Type	Défaut	Description
id	bigint (PK)	—	—
employee_id	bigint	—	FK → users.id
annuel_total	integer	30	Jours annuels attribués
annuel_pris	integer	0	Jours annuels consommés
maladie_total	integer	10	Jours maladie attribués
maladie_pris	integer	0	Jours maladie consommés
exceptionnel_total	integer	5	Jours exceptionnels attribués
exceptionnel_pris	integer	0	Jours exceptionnels consommés
annee	integer	—	Année de référence

Index : `UNIQUE (employee_id, annee)` — un seul solde par employé et par année.

3.7 `password_resets` — Demandes de réinitialisation de mot de passe

Colonne	Type	Nullable	Description
id	bigint (PK)	non	—
user_id	bigint	non	FK → users.id (cascade)
email	varchar	non	Email de l'utilisateur (index)
token	varchar	oui	Token signé (généré après approbation admin)
statut	varchar	non (def. <code>en_attente</code>)	<code>en_attente</code> <code>approuvee</code> <code>rejetee</code> (index)
created_at	timestamp	non (useCurrent)	—
expires_at	timestamp	oui	Expiration du token

3.8 `personal_access_tokens`

Table créée par Laravel Sanctum (migration standard). **Non utilisée** : l'authentification repose sur JWT (`tymon/jwt-auth`).

4. Enums et valeurs métier

4.1 Rôles

`employe` · `chef` · `admin`

4.2 Types de demande

Code	Libellé	Champs clés
<code>conge</code>	Demande de congé	date_debut, date_fin, motif
<code>autorisation</code>	Autorisation d'absence	date_debut, motif
<code>pret</code>	Prêt / Avance sur salaire	motif, montant, duree
<code>situation</code>	Changement de situation	motif (mariage, naissance, ...)
<code>document</code>	Demande de document RH	type_document

4.3 Statuts de demande

Statut	Description
en_attente	Initial — attend chef ou admin
valide_chef	Chef a validé — attend décision admin
approuvee	Admin a validé après chef
approuvee_direct	Admin a validé sans passage chef
refusee	Refusée (par chef ou admin)

4.4 Motifs de congé

Congé annuel · Congé maladie · Congé exceptionnel · Congé sans solde

4.5 Motifs de prêt

Prêt personnel · Avance sur salaire

4.6 Types de document

Attestation de travail · Attestation de salaire · Bulletin de paie · Certificat de présence

4.7 Soldes de congés — défauts annuels

- **30 jours** annuels
- **10 jours** maladie
- **5 jours** exceptionnels

5. Règles métier

5.1 Workflow de validation

```
Type = conge | autorisation
└─ Employé → Chef (valide_chef) → Admin (approuvee)
   Si chef absent / pas de chef : Admin (approuvee_direct)

Type = pret | situation | document
└─ Employé → Admin direct (approuvee_direct ou approuvee)
```

5.2 Justificatif (pièce jointe)

Obligatoire si :

- `type = situation` (tout motif)
- `type = conge` ET motif ∈ { `Congé maladie` , `Congé exceptionnel` }
- `type = pret` ET motif = `Prêt personnel`

Formats acceptés : **PDF, JPG, JPEG, PNG** · Taille max : **5 Mo**. Accès au fichier : **auteur + chef direct + admin** (contrôle dans `DemandeController::telechargerJustificatif`).

5.3 Dates

- `date_debut >= today` (validé côté client + serveur avec `after_or_equal:today`)
- `date_fin >= date_debut`

5.4 Décrément du solde de congés

Lorsqu'une demande de type `conge` passe en `approuvee` / `approuvee_direct` , le nombre de jours entre `date_debut` et `date_fin` est ajouté à la bonne colonne `_pris` de `conges_soldes` selon le motif (annuel / maladie / exceptionnel).

5.5 Document signé

Pour `type = document` approuvé, un PDF est généré à la volée avec **DomPDF** portant un QR code qui encode un hash **HMAC-SHA256** vérifiable publiquement via `GET /api/verify?...` .

6. API REST

Base URL : `http://localhost:8000/api`

6.1 Routes publiques

Verbe	URL	Contrôleur@méthode	Description
POST	/login	AuthController@login	Authentification → retourne JWT
POST	/register	AuthController@register	Inscription
POST	/forgot-password	PasswordResetController@request	Demander un reset
GET	/reset-password/validate	PasswordResetController@validateToken	Valider un token
POST	/reset-password	PasswordResetController@reset	Appliquer un reset
GET	/verify	DocumentVerificationController@verify	Vérifier PDF signé (QR)

6.2 Routes protégées (auth:api)

Header requis : `Authorization: Bearer <token>`

Auth

Verbe	URL	Méthode	Description
POST	/logout	AuthController@logout	Invalide le token
GET	/me	AuthController@me	Utilisateur courant
POST	/broadcasting/auth	Broadcast::auth	Auth WebSocket Reverb

Demandes

Verbe	URL	Méthode	Rôles	Description
GET	/demandes	DemandeController@index	tous	Liste filtres par rôle
POST	/demandes	DemandeController@store	tous	Créer une demande (multiple data)
GET	/demandes/{id}	DemandeController@show	tous	Détail d'une demande
PUT	/demandes/{id}	DemandeController@update	chef/admin	Valider ou refuser une demande
DELETE	/demandes/{id}	DemandeController@destroy	auteur/admin	Supprimer une demande
GET	/demandes/{id}/telecharger	DemandeController@telecharger	auteur/admin	PDF signé (document)
GET	/demandes/{id}/justificatif	DemandeController@telechargerJustificatif	auteur/chef/admin	Justificatif de dépôt uploadé

Notifications

Verbe	URL	Méthode	Description
GET	/notifications	NotificationController@index	Mes notifications
PUT	/notifications/{id}/lu	NotificationController@marquerLu	Marquer lue
PUT	/notifications/tout-lu	NotificationController@marquerToutLu	Tout marquer

Messages

Verbe	URL	Méthode	Description
GET	/messages	MessageController@index	Tous mes messages
GET	/messages/contacts	MessageController@contacts	Liste contacts
GET	/messages/non-lus	MessageController@nonLus	Compteur non-lus
GET	/messages/conversation/{userId}	MessageController@conversation	Thread avec un user
POST	/messages	MessageController@store	Envoyer
PUT	/messages/{id}/lu	MessageController@marquerLu	Marquer lu

Soldes de congés

Verbe	URL	Méthode	Rôles	Description
GET	/users/{id}/conges	CongesController@show	soi/chef/admin	Solde de l'année
PUT	/users/{id}/conges	CongesController@update	admin	Ajustement manuel

Users

Verbe	URL	Méthode	Rôles	Description
GET	/users	UserController@index	chef/admin	Liste
GET	/users/dashboard	UserController@dashboard	admin	Stats globales
GET	/me/stats	UserController@meStats	tous	Stats personnelles
GET	/users/{id}	UserController@show	soi/admin	Détail
PUT	/users/{id}	UserController@update	soi/admin	Modification
DELETE	/users/{id}	UserController@destroy	admin	Suppression
POST	/users/{id}/photo	UserController@uploadPhoto	soi/admin	Upload avatar
DELETE	/users/{id}/photo	UserController@deletePhoto	soi/admin	Supprimer avatar

Chatbot

Verbe	URL	Méthode	Description
POST	/chatbot	ChatbotController@chat	Gemini AI (thinkingBudget:0, maxOutputTokens:2048)

Password Resets (admin)

Verbe	URL	Méthode	Description
GET	/password-resets	PasswordResetController@adminList	Liste des demandes
PUT	/password-resets/{id}/approve	PasswordResetController@adminApprove	Générer token
PUT	/password-resets/{id}/reject	PasswordResetController@adminReject	Rejeter

7. Modèles Eloquent

7.1 User (implements JWTSubject)

```
$fillable =
['nom', 'prenom', 'email', 'password', 'role', 'chef_id', 'departement', 'poste', 'telephone', 'photo'];
$hidden = ['password', 'remember_token'];
$appends = ['photo_url'];
$casts = ['email_verified_at' => 'datetime', 'password' => 'hashed'];
```

Relations : chef() belongsTo · employes() hasMany · congésSolde() hasOne · demandes()

hasMany **Accesseur :** photo_url → asset('storage/' . \$photo) ou null **JWT :**

getJWTIdentifier(), getJWTCustomClaims()

7.2 Demande

```
$fillable = [
    'employee_id', 'chef_id', 'admin_id', 'type', 'statut',
    'date_debut', 'date_fin', 'montant', 'duree', 'motif', 'type_document',
    'commentaire', 'commentaire_chef', 'commentaire_admin', 'piece_jointe',
];
$casts = ['date_debut' => 'date', 'date_fin' => 'date', 'montant' => 'decimal:2'];
$appends = ['piece_jointe_url'];
```

Relations : employee(), chef(), admin() (tous belongsTo User)

7.3 Notification

```
public $timestamps = false;    // Δ pas d'updated_at
$fillable = ['user_id','demande_id','message','lu'];
```

Relations : `user()` , `demande()`

7.4 Message

```
public $timestamps = false;
$fillable = ['expediteur_id','destinataire_id','contenu','lu'];
```

Relations : `expediteur()` , `destinataire()`

7.5 CongesSolde

```
$fillable = ['employee_id','annuel_total','annuel_pris','maladie_total','maladie_pris',
            'exceptionnel_total','exceptionnel_pris','annee'];
public $timestamps = false;
```

Relation : `employee()` belongsTo User

7.6 Document

Table de référence pour les fichiers générés (PDF signés). Relations : `demande()` , `employee()` .

7.7 PasswordReset

```
$fillable = ['user_id','email','token','statut','expires_at'];
public $timestamps = false;
```

Relation : `user()` belongsTo User

8. Frontend — pages et composants

8.1 Pages (`src/pages/`)

Fichier	Route	Rôles	Description
Landing.jsx	/	public	Page d'accueil ArabSoft
Login.jsx	/login	public	Connexion JWT
ForgotPassword.jsx	/forgot-password	public	Demande de reset
ResetPassword.jsx	/reset-password	public	Saisie nouveau mot de passe
VerifyDocument.jsx	/verify	public	Vérification d'authenticité d'un PDF (QR)
Dashboard.jsx	/dashboard	tous	Stats + demandes récentes
Demandes.jsx	/demandes	employé	Mes demandes (CRUD + upload justificatif)
AllDemandes.jsx	/all-demandes	chef/admin	Toutes les demandes + validation
Employes.jsx	/employes	chef/admin	Liste et gestion employés
Messages.jsx	/messages	tous	Messagerie interne temps réel
Profil.jsx	/profil	tous	Mon profil + photo
Chatbot.jsx	/chatbot	tous	Assistant Gemini AI
PasswordResets.jsx	/password-resets	admin	Gestion demandes de reset

8.2 Composants partagés (`src/components/`)

Fichier	Rôle
Layout.jsx	Sidebar + Topbar + Outlet
Sidebar.jsx	Navigaton filtrée par rôle
Topbar.jsx	Titre + cloche notifications (WebSocket)
PrivateRoute.jsx	Redirige vers <code>/login</code> si non authentifié
ChatbotWidget.jsx	Bulle flottante Gemini sur toutes les pages

8.3 Contexte & HTTP

- `src/context/AuthContext.jsx` — `{ user, token, login(), logout() }`, persiste dans `localStorage`
- `src/api/axios.js` — instance Axios avec `baseURL` + interceptor JWT + handler 401 → logout

9. Fichiers et stockage

Le disque Laravel `public` (`storage/app/public/`) est lié à `public/storage/` via `php artisan storage:link` .

Dossier	Contenu	Nommage
<code>storage/app/public/avatars/</code>	Photos de profil	<code>{user_id}_{timestamp}.{ext}</code>
<code>storage/app/public/demandes/</code>	Justificatifs (PDF/JPG/PNG)	<code>{demande_id}_{timestamp}.{ext}</code>

Accès public : `asset('storage/avatars/...')` → URL directe
Accès contrôlé justificatifs : route `GET /api/demandes/{id}/justificatif` (vérifie auteur/chef/admin) — **recommandé pour production**

9.1 PDF signés

Les documents administratifs (`type = document` approuvés) sont générés à la volée avec **DomPDF** et portent un QR code contenant un payload :

```
id={demande_id}&h={hmac_sha256(id|employee_id|type_document, APP_KEY)}
```

Vérification publique via `GET /api/verify?id=...&h=...` .

10. Intégrations externes

10.1 JWT (tymon/jwt-auth 2.3.0)

- Guard `api` dans `config/auth.php` avec driver `jwt`
- `User` implémente `JWTSubject`
- TTL configurable via `.env` (`JWT_TTL`)

10.2 Gemini 2.5 Flash

- Endpoint : `POST /api/chatbot`
- Configuration : `thinkingConfig.thinkingBudget = 0` , `maxOutputTokens = 2048`
- Clé API : `GEMINI_API_KEY` dans `.env` (free tier Google AI Studio)
- Contexte injecté : infos de l'utilisateur connecté (solde congés, demandes récentes)

10.3 Reverb WebSocket

- Driver broadcast : `reverb`
 - Channel privé : `App.Models.User.{id}` (auth via `/broadcasting/auth`)
 - Événements diffusés : `NotificationCreated` , `MessageSent`
-

11. Seeders et données de test

- `database/factories/UserFactory.php` — fabrique générique (nom, email, password hashé)
- `database/seeders/DatabaseSeeder.php` — crée un utilisateur de test (`test@example.com`)

Pour peupler la base :

```
php artisan db:seed
```

12. Variables d'environnement

12.1 Backend (`.env`)

```
APP_NAME="Portail RH ArabSoft"
APP_ENV=local
APP_KEY=base64:...          # php artisan key:generate
APP_URL=http://localhost:8000

DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=portail_rh
DB_USERNAME=postgres
DB_PASSWORD=rayen

JWT_SECRET=...              # php artisan jwt:secret
JWT_TTL=1440                 # minutes (24h)

BROADCAST_DRIVER=reverb
REVERB_APP_ID=...
REVERB_APP_KEY=...
REVERB_APP_SECRET=...
REVERB_HOST=localhost
REVERB_PORT=8080

GEMINI_API_KEY=...          # Google AI Studio
FILESYSTEM_DISK=public
QUEUE_CONNECTION=database
```

12.2 Frontend (`.env`)

```
VITE_API_URL=http://localhost:8000/api
VITE_REVERB_APP_KEY=...
VITE_REVERB_HOST=localhost
VITE_REVERB_PORT=8080
VITE_REVERB_SCHEME=http
```

Annexe — Commandes utiles

Backend

```
cd C:\Users\DELL\Documents\projetpfe\portail-rh-backend
php artisan serve                # Démarre (port 8000)
php artisan migrate               # Exécute les migrations
php artisan migrate:status        # État des migrations
php artisan db:seed               # Seed la BDD
php artisan jwt:secret            # Génère JWT_SECRET
php artisan storage:link          # Crée symlink public/storage
php artisan reverb:start          # Démarre WebSocket
php artisan route:list            # Liste toutes les routes
```

Frontend

```
cd C:\Users\DELL\Documents\projetpfe\portail-rh-frontend
npm install
npm run dev                       # Port 5173
npm run build                     # Build production
```

Fin du document.